MANUAL TÉCNICO

PROYECTO 2 ANALIZADOR KANBAN

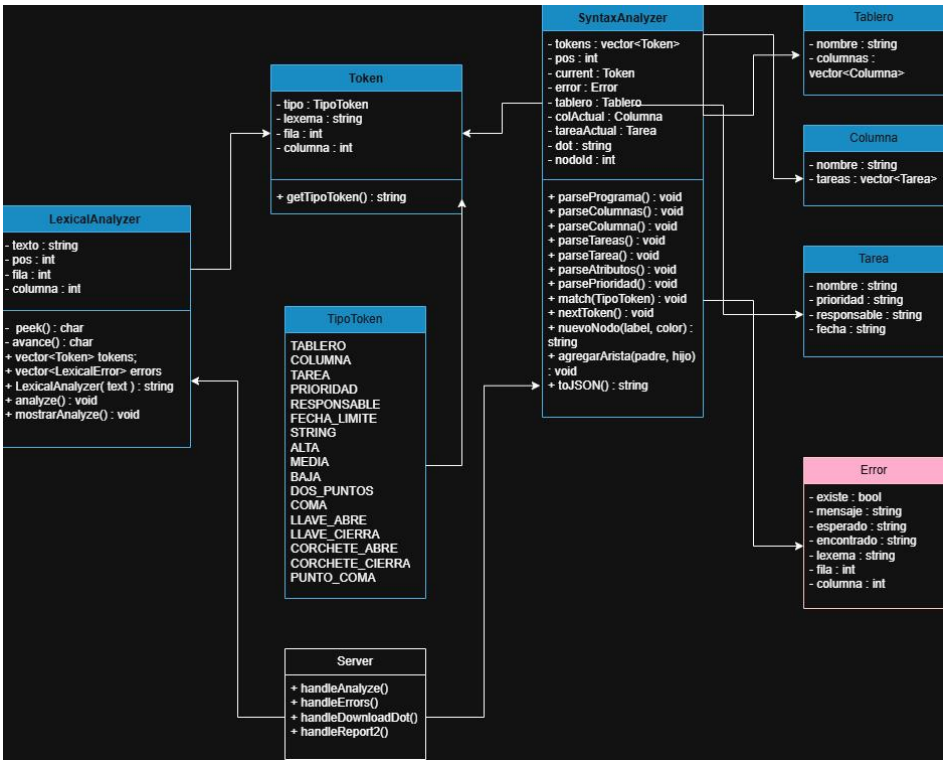
INTRODUCCIÓN

Este sistema es un analizador léxico y sintáctico desarrollado en C++ cuyo objetivo es interpretar un lenguaje propio tipo Kanban, construir una estructura de datos jerárquica y generar representaciones visuales y reportes automáticos. El sistema implementa técnicas de compiladores:

- Análisis léxico (AFD)
- Análisis sintáctico (parser descendente recursivo)
- Generación de árbol de derivación (.DOT)
- Manejo de errores léxicos y sintácticos
- Generación de reportes estructurados

ARQUITECTURA DEL SISTEMA

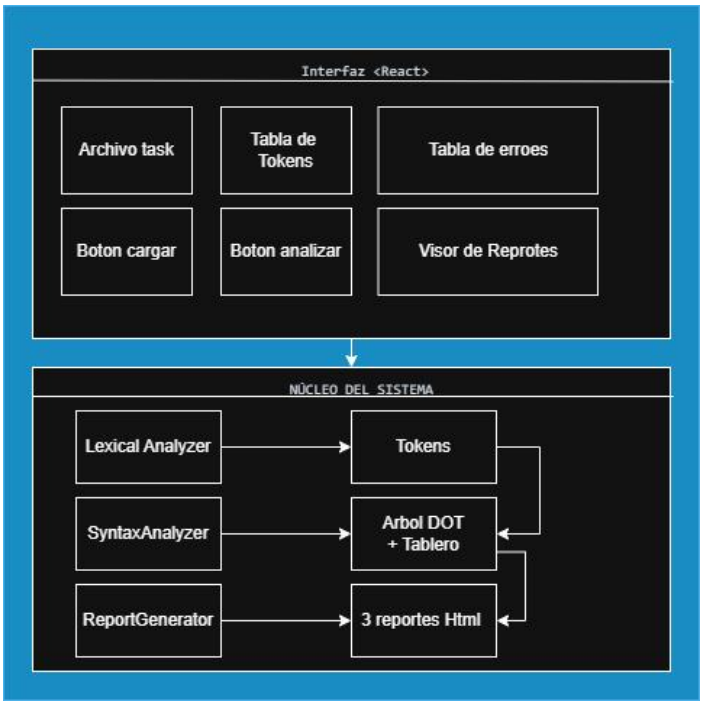
El sistema está dividido en partes bien separadas para que todo sea más ordenado, fácil de mantener y para poder crecer sin que se vuelva un desastre.



Relaciones entre Clases

Clase Origen	Clase Destino	Relación	Multiplicidad
LexicalAnalyzer	Token	Composición	1 : N
LexicalAnalyzer	LexicalError	Composición	1 : N
SyntaxAnalyzer	Token	Asociación	1 : N
SyntaxAnalyzer	Tablero	Composición	1 : 1
SyntaxAnalyzer	SyntaxError	Asociación	1 : 1
Tablero	Columna	Composición	1 : N
Columna	Tarea	Composición	1 : N

Diagrama de Componentes



Alcance

El analizador es capaz de:

- Realizar un análisis léxico basado en un Autómata Finito Determinista (AFD).
- Ejecutar un análisis sintáctico mediante un parser descendente recursivo.
- Construir un árbol de derivación en formato DOT (Graphviz).
- Detectar y reportar errores léxicos y sintácticos con ubicación precisa.

- Integrarse con un frontend web (React) a través de una API en C++.

Aplicaciones Prácticas

- Gestión de proyectos visuales.
- Validación de configuraciones Kanban.
- Generación de reportes automáticos a partir de lenguaje natural estructurado.

Estructura del Lenguaje TaskScript (Según Enunciado)

```
TABLERO "Nombre del Proyecto" {
    COLUMNA "Por Hacer" {
        tarea: "Diseñar AFD" {prioridad: ALTA, responsable: "Jorge", fecha_limite: 2026-05-01},
        tarea: "Implementar Lexer" {prioridad: ALTA, responsable: "María", fecha_limite: 2026-05-08}
    };
    COLUMNA "En Progreso" {
        tarea: "Diseñar GUI" {prioridad: MEDIA, responsable: "Ana", fecha_limite: 2026-05-05}
    };
};
```

Flujo de Procesamiento

- Entrada: Archivo .task cargado por el usuario
- Análisis Léxico: LexicalAnalyzer::analyze() recorre el texto carácter por carácter, reconoce tokens y registra errores léxicos
- Análisis Sintáctico: SyntaxAnalyzer consume los tokens y valida la estructura.
- Construcción de Datos: Se llenan las estructuras Tablero, Columna, Tarea
- Generación de Reportes: Se crean 3 archivos HTML y el archivo .dot
- Visualización: La GUI muestra tokens, errores y permite abrir los reportes

Características clave:

- Palabras reservadas en MAYÚSCULAS: TABLERO, COLUMNA
- Palabras reservadas en minúsculas: tarea, prioridad, responsable, fecha_limite
- Atributos dentro de llaves {} (NO corchetes)
- Separación de tareas por coma
- Cada columna termina con punto y coma ;

Tabla de transiciones AFD

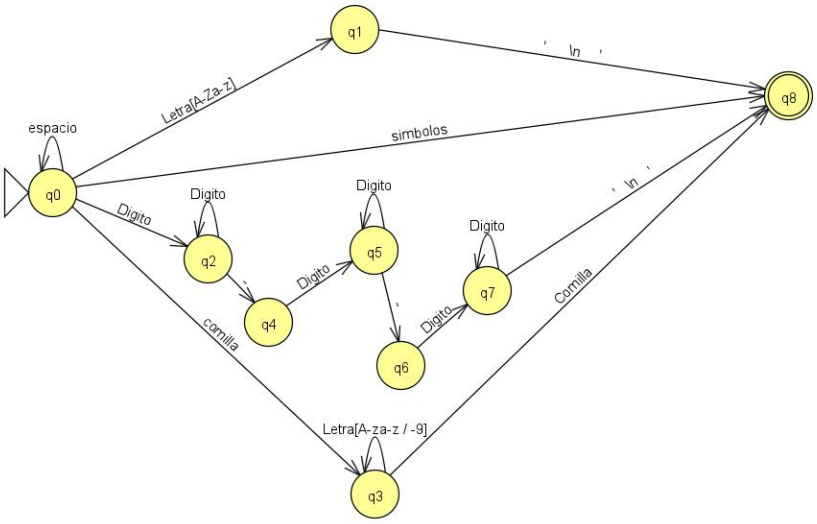
Desde	Entrada	Hacia	Token generado
q0	espacio, tab, \n	q0	-

Desde	Entrada	Hacia	Token generado
q0	letra (a-z, A-Z)	q1	-
q0	dígito (0-9)	q2	-
q0	"	q3	-
q0	{	ACEPTAR	LLAVE_ABRE
q0	}	ACEPTAR	LLAVE_CIERRA
q0	[	ACEPTAR	CORCHETE_ABRE
q0	]	ACEPTAR	CORCHETE_CIERRA
q0	:	ACEPTAR	DOS_PUNTOS
q0	,	ACEPTAR	COMA
q0	;	ACEPTAR	PUNTO_COMA
q0	otro carácter	ERROR	carácter no reconocido
q1	letra, dígito	q1	-
q1	espacio, símbolo, fin	ACEPTAR	TABLERO, COLUMNA, tarea, prioridad, responsable, fecha_limite, ALTA, MEDIA, BAJA, o ERROR
q2	dígito	q2	-
q2	-	q4	-
q2	espacio, símbolo, fin	ACEPTAR	ENTERO
q3	cualquier carácter (excepto ", \n, \0)	q3	-
q3	"	ACEPTAR	STRING
q3	\n o \0	ERROR	cadena no cerrada
q4	dígito, -	q4	-
q4	espacio, símbolo, fin	ACEPTAR	FECHA o ERROR

Tabla de Tokens Reconocidos

Categoría	Tokens	Ejemplo
Palabras reservadas (MAYÚSCULAS)	TABLERO, COLUMNA	TABLERO, COLUMNA
Palabras reservadas (minúsculas)	tarea, prioridad, responsable, fecha_limite	tarea:, prioridad:
Prioridades	ALTA, MEDIA, BAJA	ALTA
Literales	STRING, ENTERO, FECHA	"texto", 123, 2026-05-01
Símbolos	{ } [ ] : , ;	{, }, [,], :, ,

Automata Determinista Finito



Gramatica Libre de Contexto

<programa> ::= TABLERO STRING "{" <columnas> "}" ";"

<columnas> ::= <columna> <columnas_aux>

<columnas_aux> ::= <columna> <columnas_aux> | ε

<columna> ::= COLUMNA STRING "{" <tareas> "}" ";"

<tareas> ::= <tarea> <tareas_aux>

<tareas_aux> ::= "," <tarea> <tareas_aux> | ε

<tarea> ::= TAREA ":" STRING "[" <atributos> "]"

<atributos> ::= <atributo> <atributos_aux>

<atributos_aux> ::= "," <atributo> <atributos_aux> | ε

<atributo> ::= PRIORIDAD ":" <nivel>

| RESPONSABLE ":" STRING

| FECHA_LIMITE ":" FECHA

<nivel> ::= ALTA | MEDIA | BAJA

DESCRIPCIÓN DEL PARSER

El parser implementado es de tipo descendente recursivo, construido manualmente en C++. Cada regla de la gramática tiene una función correspondiente que valida la estructura del código fuente.
Funciones principales del parser

Función	Regla gramatical	Descripción
parsePrograma()	<programa>	Valida la raíz del archivo y crea el nodo raíz del árbol DOT
parseColumnas()	<columnas>	Itera sobre todas las columnas del tablero
parseColumna()	<columna>	Valida una columna y guarda su información
parseTareas()	<tareas>	Itera sobre todas las tareas de una columna
parseTarea()	<tarea>	Valida una tarea y guarda sus atributos
parseAtributos()	<atributos>	Itera sobre todos los atributos de una tarea
parseAtributo()	<atributo>	Valida prioridad, responsable o fecha límite

Función	Regla gramatical	Descripción
parsePrioridad()	<nivel>	Valida ALTA, MEDIA o BAJA

Funciones auxiliares

Función	Descripción
match(TipoToken)	Compara el token actual con el esperado. Si coincide, avanza; si no, registra error
nextToken()	Avanza al siguiente token en la lista
nuevoNodo()	Crea un nodo en el árbol de derivación (formato DOT)
agregarArista()	Conecta dos nodos en el árbol de derivación
toJSON()	Genera un objeto JSON con toda la información del tablero

Ejemplo de funcionamiento

Para la entrada:

TABLERO "Mi Proyecto" { COLUMNA "Tareas" { tarea: "Estudiar" [prioridad: ALTA] }; };

El flujo del parser es:

parsePrograma() : reconoce TABLERO, guarda el nombre, encuentra {

parseColumnas() : llama a parseColumna()

parseColumna() : reconoce COLUMNA, guarda el nombre, encuentra {

parseTareas() : llama a parseTarea()

parseTarea() : reconoce tarea:, guarda el título, encuentra [

parseAtributos() : llama a parseAtributo()

parseAtributo() : reconoce prioridad:, llama a parsePrioridad()

parsePrioridad() : reconoce ALTA y guarda el valor

Se cierran corchetes, llaves y punto y coma

Justificacion de Diseño

Decisión	¿Por qué?
C++ como lenguaje	Alto rendimiento, control de memoria, ideal para compiladores
Parser descendente recursivo	Traducción 1:1 desde la gramática, fácil de implementar y depurar
AFD manual (sin regex)	Permite trackear línea/columna exacta. El enunciado lo exige
Corchetes [ ] para atributos	Diferencia visual entre bloques ({}) y propiedades ([])
Graphviz (DOT)	Formato estándar para visualizar árboles de derivación
Estructura Tablero→Columna→Tarea	Representa fielmente la realidad de un tablero Kanban
Detección de errores inmediata	Simple y efectiva. El primer error suele causar los siguientes

Manual de Compilacion

Requisitos previos

Herramienta	Versión	Para qué
C++ Compiler	C++17 o superior	Compilar el proyecto
CMake	3.16+	Gestión de la compilación

Herramienta	Versión	Para qué
React / Nodo.js		Interfaz gráfica
Graphviz	Cualquiera	Generar imágenes del árbol DOT

Estrategia de Recuperacion de Errores

Enfoque implementado

El parser utiliza una detección de errores con detención inmediata. Cuando se encuentra un error, el análisis se detiene y se registra la información completa del error.

Mecanismo de match()

```
void SyntaxAnalyzer::match(TipoToken esperado) {
    if (error.existe) return;  // Si ya hay error, no continuar

    if (current.tipo == esperado) {
        nextToken();  // Token correcto, avanzar
    } else {
        error.existe = true;
        error.esperado = tipoToString(esperado);
        error.encontrado = current.getTipoToken();
        error.lexema = current.lexema;
        error.fila = current.fila;
        error.columna = current.columna;
        error.mensaje = "Se esperaba '" + error.esperado +
                        "' pero se encontró '" + error.encontrado + "'";
    }
}
```

Información registrada en cada error

Campo	Tipo	Descripción
existe	bool	Indica si ocurrió un error
mensaje	string	Descripción legible del error
esperado	string	Token que se esperaba
encontrado	string	Token que se encontró
lexema	string	Texto del token encontrado
fila	int	Línea donde ocurrió

Campo	Tipo	Descripción
columna	int	Columna donde ocurrió